

❖ 1. Variables in C++ (Java ke jaisa)

Java Type	C++ Equivalent
Local Variable	Function ke andar declare hota hai
Instance Variable	Class ke andar declare, non-static member
Static Variable	<code>static</code> keyword se class level variable

❖ LOCAL vs GLOBAL VARIABLES IN C++

✓ 1. Local Variable (Function ke andar banaye jaate hain)

✦ Definition:

Local variable wo variable hota hai jo kisi function, loop ya block ke andar banaya jaata hai. Iska scope sirf usi block tak hota hai. Bahar use nahi kar sakte.

🗑 Memory: Function chalte hi banta hai, aur function khatam hote hi destroy ho jaata hai.

❖ Example:

```
#include <iostream>
using namespace std;

void printNumber() {
    int x = 10; // local variable
    cout << "Local x = " << x << endl;
}

int main() {
    printNumber();
    // cout << x; // ✗ Error: x not accessible here
    return 0;
}
```

🖨 Output:

```
Local x = 10
---
```

✓ 2. Global Variable (Function ke bahar declare kiye jaate hain)

✦ Definition:

Global variable wo hota hai jo sabhi functions ke bahar declare kiya jaata hai. Ye pura program me kahin bhi accessible hota hai (agar scope me ho).

🗑 Memory: Program start hote hi banta hai, aur program end hone tak memory me rehta hai.

❖ Example:

```
#include <iostream>
using namespace std;
```

```
int x = 100; // global variable
```

```
void show() {  
    cout << "Global x in show(): " << x << endl;  
}
```

```
int main() {  
    cout << "Global x in main(): " << x << endl;  
    show();  
    return 0;  
}
```

🖨️ Output:

```
Global x in main(): 100  
Global x in show(): 100  
---
```

🔄 Difference Table:

Feature	Local Variable	Global Variable
Kahan banta hai?	Function/block ke andar	Function ke bahar
Scope	Sirf block ke andar	Pura program me access hota hai
Memory lifetime	Function ke time tak	Program end tak
Access outside block	✗ Nahi ho sakta	✓ Ho sakta hai

💡 Note:

- Agar local aur global variable dono ka naam same ho to local variable ko priority milti hai.
- Aap `::` operator se global variable ko forcefully access kar sakte ho.

💎 Example:

```
#include <iostream>  
using namespace std;
```

```
int x = 50; // global
```

```
int main() {  
    int x = 10; // local  
    cout << "Local x = " << x << endl; // Output: 10  
    cout << "Global x = " << ::x << endl; // Output: 50  
    return 0;  
}
```

➔ Static Variable : ➔

```
#include <iostream>
using namespace std;
void counter() {
    static int count = 0; // static local variable
    count++;
    cout << "Count: " << count << endl;
}

int main() {
    counter(); // Output: Count: 1
    counter(); // Output: Count: 2
    counter(); // Output: Count: 3
    return 0;
}
```

main.cpp	Run	Output
<pre>1 #include <iostream> 2 using namespace std; 3 void counter() { 4 static int count = 0; // static local variable 5 count++; 6 cout << "Count: " << count << endl; 7 } 8 9 int main() { 10 counter(); // Output: Count: 1 11 counter(); // Output: Count: 2 12 counter(); // Output: Count: 3 13 return 0; 14 } 15</pre>		Count: 1 Count: 2 Count: 3 === Code Executed

➔ OOPS

In C++, inheritance allows one class (the derived class) to inherit properties and behaviors (members and methods) from another class (the base class). There are several types of inheritance in C++:

1. Single Inheritance:

In single inheritance, a derived class inherits from a single base class.

Example:

```
class Base {
public:
    void show() {
        std::cout << "Base class" << std::endl;
    }
};

class Derived : public Base {
public:
    void display() {
        std::cout << "Derived class" << std::endl;
    }
};

int main() {
    Derived obj;
    obj.show();    // Base class method
    obj.display(); // Derived class method
    return 0;
}
```

2. Multiple Inheritance:

In multiple inheritance, a derived class inherits from more than one base class.

Example:

```
class Base1 {
public:
    void show() {
        std::cout << "Base1 class" << std::endl;
    }
};

class Base2 {
public:
    void display() {
        std::cout << "Base2 class" << std::endl;
    }
};

class Derived : public Base1, public Base2 {
public:
    void showMessage() {
        std::cout << "Derived class" << std::endl;
    }
};

int main() {
    Derived obj;
```

```

    obj.show();          // Base1 class method
    obj.display();       // Base2 class method
    obj.showMessage();   // Derived class method
    return 0;
}

```

3. Multilevel Inheritance:

In multilevel inheritance, a derived class is derived from another derived class (i.e., there is a chain of inheritance).

Example:

```

class Base {
public:
    void show() {
        std::cout << "Base class" << std::endl;
    }
};

class Derived1 : public Base {
public:
    void display() {
        std::cout << "Derived1 class" << std::endl;
    }
};

class Derived2 : public Derived1 {
public:
    void showMessage() {
        std::cout << "Derived2 class" << std::endl;
    }
};

int main() {
    Derived2 obj;
    obj.show();          // Base class method
    obj.display();       // Derived1 class method
    obj.showMessage();   // Derived2 class method
    return 0;
}

```

4. Hierarchical Inheritance:

In hierarchical inheritance, multiple derived classes inherit from a single base class.

Example:

```

class Base {
public:
    void show() {

```

```

        std::cout << "Base class" << std::endl;
    }
};

class Derived1 : public Base {
public:
    void display1() {
        std::cout << "Derived1 class" << std::endl;
    }
};

class Derived2 : public Base {
public:
    void display2() {
        std::cout << "Derived2 class" << std::endl;
    }
};

int main() {
    Derived1 obj1;
    Derived2 obj2;
    obj1.show();
    obj1.display1();
    obj2.show();
    obj2.display2();
    return 0;
}

```

5. Hybrid Inheritance:

Hybrid inheritance is a combination of two or more types of inheritance. For example, combining multiple and multilevel inheritance.

Example:

```

class Base {
public:
    void show() {
        std::cout << "Base class" << std::endl;
    }
};

class Derived1 : public Base {
public:
    void display1() {
        std::cout << "Derived1 class" << std::endl;
    }
};

class Derived2 {
public:
    void display2() {
        std::cout << "Derived2 class" << std::endl;
    }
};

```

```

class FinalDerived : public Derived1, public Derived2 {
public:
    void showMessage() {
        std::cout << "FinalDerived class" << std::endl;
    }
};

int main() {
    FinalDerived obj;
    obj.show();           // Base class method
    obj.display1();       // Derived1 class method
    obj.display2();       // Derived2 class method
    obj.showMessage();    // FinalDerived class method
    return 0;
}

```

6. Virtual Inheritance:

Virtual inheritance is used to solve the problem of *diamond problem* in multiple inheritance. When two or more base classes share a common ancestor, virtual inheritance ensures that the shared base class is only inherited once.

Example:

```

class Base {
public:
    void show() {
        std::cout << "Base class" << std::endl;
    }
};

class Derived1 : virtual public Base {
public:
    void display1() {
        std::cout << "Derived1 class" << std::endl;
    }
};

class Derived2 : virtual public Base {
public:
    void display2() {
        std::cout << "Derived2 class" << std::endl;
    }
};

class FinalDerived : public Derived1, public Derived2 {
public:
    void showMessage() {
        std::cout << "FinalDerived class" << std::endl;
    }
};

int main() {
    FinalDerived obj;
}

```

```

    obj.show();           // Base class method (only one call due to virtual
inheritance)
    obj.display1();       // Derived1 class method
    obj.display2();       // Derived2 class method
    obj.showMessage();    // FinalDerived class method
    return 0;
}

```

Summary:

OOPs Concepts in C++ (Simplified)

C++ me OOPs ke **4 main pillars** hote hain:

OOP Concept	Key Idea	Keyword/Example
Encapsulation	Data hiding	<code>private</code> , getters/setters
Abstraction	Hiding implementation details	Only show <code>public</code> methods
Inheritance	Reuse code	<code>class B : public A</code>
Polymorphism	Many forms of same function	Overloading / Overriding

Encapsulation in C++

 Definition:

Encapsulation ka matlab hai:

“Data (variables) aur functions (methods) ko ek single unit (class) me bundle karna.”

Aur data ko **private** bana kar **direct access** se **protect** karna.

CODE ::=>

main.cpp	Output
<pre>1 #include <iostream> 2 using namespace std; 3 class Student { 4 private: 5 int age; // Private: directly access nahi kar sakte 6 public: 7 void setAge(int a) { 8 age = a; 9 } 10 int getAge() { 11 return age; 12 } 13 }; 14 int main() { 15 Student s; 16 s.setAge(18); // Set value through 17 cout << "Student Age: " << s.getAge(); // Get value through 18 return 0; 19 } 20</pre>	<pre>Student Age: 18 === Code Executing ===</pre>

CODE ➔

PRESS ME

PRESS ME (Example 2)

Abstraction in C++

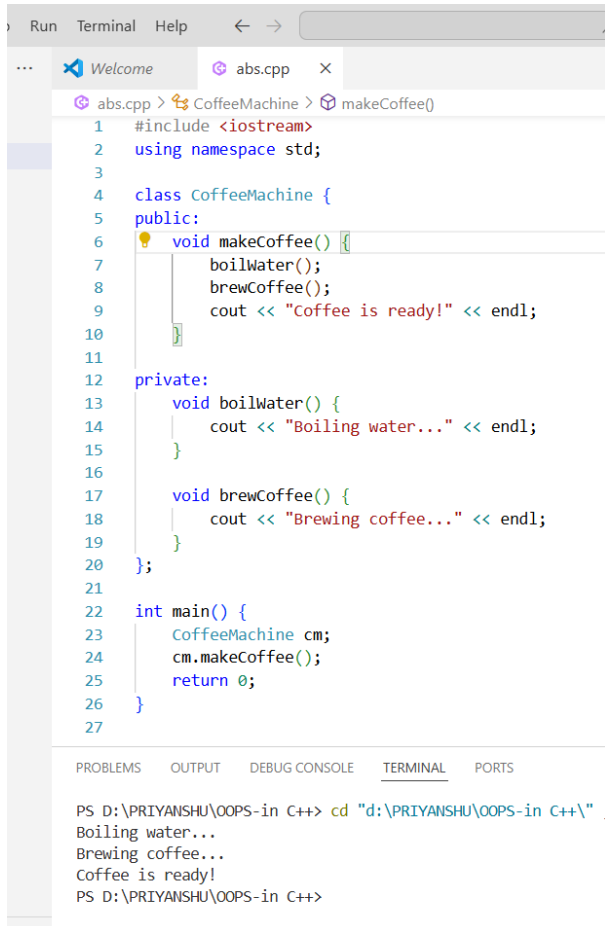
🔗 Definition (Asaan Bhasha Mein):

Abstraction ka matlab hai:

"User ko sirf zaroori cheezein dikhana, aur baaki sab details chhupa lena."

Yaani:

- User ko **function ka kaam** dikhai de
- **Function ke andar kya chal raha hai**, wo **hide** ho



```
1 #include <iostream>
2 using namespace std;
3
4 class CoffeeMachine {
5 public:
6     void makeCoffee() {
7         boilWater();
8         brewCoffee();
9         cout << "Coffee is ready!" << endl;
10    }
11 private:
12     void boilWater() {
13         cout << "Boiling water..." << endl;
14     }
15     void brewCoffee() {
16         cout << "Brewing coffee..." << endl;
17     }
18 };
19
20 int main() {
21     CoffeeMachine cm;
22     cm.makeCoffee();
23     return 0;
24 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS D:\PRIYANSHU\OOPS-in C++> cd "d:\PRIYANSHU\OOPS-in C++\" ;
Boiling water...
Brewing coffee...
Coffee is ready!
PS D:\PRIYANSHU\OOPS-in C++>

CODE ➔

PRESS ME

Polymorphism in C++

🔗 Definition:

Polymorphism ka matlab hai:

“Ek function ya object ka multiple forms me behave karna.”

Yaani, ek function ya operator alag-alag situations me alag kaam kare.

🔥 Types of Polymorphism:

1. **Compile-time Polymorphism** (Static binding)

- Function Overloading

2. **Run-time Polymorphism** (Dynamic binding)

- Function Overriding (Virtual functions)

1 **Compile-time Polymorphism**

a) Function Overloading:

Ek hi naam ke functions, lekin alag parameters ke saath.

CODE →

```
#include <iostream>

using namespace std;

class Calculator {
public:
    int add(int a, int b) {
        return a + b;
    }

    float add(float a, float b) {
        return a + b;
    }
};

int main() {
    Calculator calc;

    cout << calc.add(5, 3) << endl;    // Output: 8

    cout << calc.add(2.5f, 3.5f) << endl; // Output: 6.0

    return 0;
}
```

PRESS ME →

2 Run-time Polymorphism

Function Overriding (with virtual function):

Derived class apne hisaab se base class ke function ko change kar sakti hai.

```
main.cpp
1 #include <iostream>
2 using namespace std;
3 class Animal {
4 public:
5     virtual void sound() {
6         cout << "Animal makes sound" << endl;
7     }
8 };
9 class Dog : public Animal {
10 public:
11     void sound() override {
12         cout << "Dog barks" << endl;
13     }
14 };
15 int main() {
16     Animal* animalPtr;
17     Dog d;
18     animalPtr = &d;
19
20     animalPtr->sound(); // Output: Dog barks (run-time polymorphism)
21     return 0;
22 }
23
```

Output: Dog barks

=== Code Executed

[PRESS ME →](#)

Polymorphism Type	How it works	Example
Compile-time	Function/Operator overloading	Same function name, different params
Run-time	Virtual functions (Overriding)	Base class pointer calls derived class function

- **Single Inheritance:** A derived class inherits from one base class.
- **Multiple Inheritance:** A derived class inherits from more than one base class.
- **Multilevel Inheritance:** A derived class inherits from another derived class.
- **Hierarchical Inheritance:** Multiple derived classes inherit from a single base class.
- **Hybrid Inheritance:** A combination of two or more inheritance types.
- **Virtual Inheritance:** Used to handle the diamond problem and ensure a base class is inherited only once.

Each type of inheritance in C++ allows you to structure your classes differently depending on the relationships between the entities.